Apprentissage fédéré (Federated Learning)

Cours de master – optimisation, hétérogénéité, vie privée, robustesse et décentralisation

M. Amine Bouchiha

Télécom SudParis

28 octobre 2026

Plan du cours

1. Fondations : motivation, taxonomie, menaces
2. Optimisation fédérée : FedAvg
3. Hétérogénéité des données (non-IID)
4. Systèmes et communication
5. Vie privée : fuites, DP, agrégation sécurisée
6. Attaques sur l'intégrité : poisoning, backdoors
7. Défenses robustes
8. FL décentralisé (DFL)
9. Équité, incitations, régulation
10. Étude de cas et exercices

Fil rouge

Entraîner un modèle commun sans centraliser les données : à quel coût en précision, en communication, en vie privée et en confiance ?

Motivation, taxonomie, acteurs et modèle de menaces

Pourquoi l'apprentissage fédéré ?

- Données réparties entre des détenteurs (hôpitaux, banques, téléphones) qui **ne peuvent ou ne veulent pas les partager** : confidentialité, réglementation (RGPD), propriété, coût de transfert.
- **Idée** : déplacer le calcul vers les données ; seuls les **mises à jour de modèle** sont échangées.
- **Définition** : un ensemble de clients entraîne collaborativement un modèle global sous la coordination d'un serveur (ou de façon pair à pair), sans que les données brutes quittent les clients.
- **Attention** : « les données ne quittent pas le client » **n'implique pas** que le protocole préserve la vie privée (module 5).

Problème formel

K clients, client k avec jeu local $\mathcal{D}_k$ de taille n_k, $n = \sum_k n_k$.

$$\min_{w \in \mathbb{R}^d} F(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w), \qquad F_k(w) = \frac{1}{n_k} \sum_{(x,y) \in \mathcal{D}_k} \ell(w; x, y)$$

- Si les $\mathcal{D}_k$ sont des échantillons i.i.d. d'une même distribution, c'est de l'apprentissage distribué classique.
- En FL, $\mathcal{D}_k$ suit une distribution P_k **différente** : F_k et F ont des minimiseurs distincts.
- **Contraintes spécifiques** : participation partielle, communication coûteuse, hétérogénéité matérielle, vie privée.

Taxonomie : cross-device vs cross-silo

	Cross-device	**Cross-silo**
Clients	Millions de téléphones, objets	2 à 100 organisations
Disponibilité	Faible, intermittente	Élevée
Données par client	Peu	Beaucoup
État du client	Sans état (*stateless*)	Avec état possible
Fiabilité	Faible	Élevée
Menace principale	Clients compromis en masse (Sybil)	Collusion, fuites inter-organisations
Exemples	Clavier prédictif	Imagerie médicale, détection de fraude

Partitionnement des données : horizontal, vertical, transfert

- **FL horizontal** : mêmes caractéristiques, échantillons différents (hôpitaux avec les mêmes examens, patients différents). Cadre principal de ce cours.
- **FL vertical** : mêmes individus, caractéristiques différentes (banque et assureur) ; nécessite un alignement d'entités (PSI) et des protocoles de calcul conjoint.
- **Federated transfer learning** : peu de recouvrement en échantillons et en caractéristiques.
- **Topologies** : étoile (serveur central), hiérarchique, pair à pair (DFL, module 8).

Acteurs et modèle de menaces

Acteur	**Modèle**	**Menaces**
Serveur	Honnête-mais-curieux ou malveillant	Inférence sur les clients, biais d'agrégation, censure
Client	Honnête, curieux ou byzantin	Poisoning, backdoor, inférence sur les autres, *free-riding*
Collusion	Serveur + clients, ou clients entre eux	Reconstruction d'une cible, contournement des défenses
Observateur externe	Écoute du canal, utilisateur du modèle	Extraction, inférence d'appartenance (modèle final)

Trois propriétés à garantir : **confidentialité** (données), **intégrité** (modèle), **disponibilité/équité** (participation, contribution).

FedAvg, dérive locale, convergence

Algorithme FedAvg (McMahan et al., 2017)

À chaque tour t :

1. Le serveur choisit un sous-ensemble S_t de clients (fraction C) et leur envoie w_t.
2. Chaque client $k \in S_t$ effectue E époques de SGD local (pas η, lots de taille B) à partir de w_t et obtient w_{t+1}^k.
3. Le serveur agrège : $w_{t+1} = \sum_{k \in S_t} \frac{n_k}{\sum_{j \in S_t} n_j} w_{t+1}^k$.

Cas particuliers : $E = 1$, un seul pas de gradient par client $\Rightarrow$ équivalent à du SGD à gros lot (FedSGD).
Paramètres clés : C, E, B, η, nombre de tours T.
Variante serveur : $w_{t+1} = w_t + \eta_s \sum_k \frac{n_k}{n_{S_t}} (w_{t+1}^k - w_t)$ (FedOpt : Adam, Yogi côté serveur).

FedAvg : pseudo-code

```
def fedavg(w, clients, T, C, E, B, lr):
    for t in range(T):
        S = sample(clients, max(1, int(C * len(clients))))
        updates, sizes = [], []
        for k in S:
            wk = copy(w)
            for epoch in range(E):
                for batch in batches(k.data, B):
                    wk = wk - lr * grad(loss, wk, batch)
            updates.append(wk); sizes.append(len(k.data))
        w = sum(n * u for n, u in zip(sizes, updates)) / sum(sizes)
    return w
```

Le serveur ne voit que w^k (ou $\Delta_k = w^k - w_t$), jamais $\mathcal{D}_k$.

Exemple chiffré d'agrégation

Trois clients, tailles $n = (100, 200, 700)$, paramètre scalaire local $w^k = (1{,}0;\ 2{,}0;\ 3{,}0)$:

$$w_{t+1} = \frac{100 \cdot 1 + 200 \cdot 2 + 700 \cdot 3}{1000} = \frac{100 + 400 + 2100}{1000} = 2{,}6$$

- Moyenne non pondérée : $(1 + 2 + 3)/3 = 2$, très différente : la pondération par n_k donne plus de poids aux gros clients.
- **Conséquence sécurité** : un client avec une grande taille déclarée pèse davantage ; la taille n_k est une valeur auto-déclarée à vérifier ou plafonner.
- Avec $\Delta_k = w^k - w_t$ et $w_t = 0$: même résultat.

Dérive des clients (*client drift*) et convergence

- Avec E pas locaux sur des F_k différents, chaque client dérive vers **son** minimiseur $w_k^\star \neq w^\star$.
- La moyenne des modèles locaux n'est pas le modèle optimal : biais proportionnel à E, η et à l'hétérogénéité $\zeta^2 = \mathbb{E}_k \|\nabla F_k(w^\star)\|^2$.
- **Résultat typique** (cas convexe, non-i.i.d.) : FedAvg avec pas constant converge vers un voisinage de l'optimum ; il faut décroître le pas pour converger exactement.
- **Compromis** : plus de calcul local réduit la communication mais augmente la dérive.
- Remèdes : FedProx, SCAFFOLD, FedNova, FedOpt (module 3).

Non-IID, FedProx, SCAFFOLD, personnalisation

Formes de non-IID

Type	Description	Exemple
Décalage des étiquettes (*label skew*)	$P_k(y)$ varie	Un hôpital voit surtout une pathologie
Décalage des caractéristiques	$P_k(x\|y)$ varie	Scanners de marques différentes
Décalage de quantité	n_k très inégaux	Petits et grands clients
Décalage temporel	Distribution évolue	Usage mobile saisonnier
Concept shift	$P_k(y\|x)$ varie	Étiquetage différent selon les sites

Simulation courante : partition de Dirichlet : pour chaque classe, répartition des échantillons entre clients selon $\mathrm{Dir}(\alpha)$; α petit (0,1) $\Rightarrow$ très hétérogène ; α grand (100) $\Rightarrow$ proche i.i.d.

FedProx et SCAFFOLD

FedProx (Li et al., 2020) : objectif local régularisé vers le modèle global :

$$\min_{w} \; F_k(w) + \frac{\mu}{2}\|w - w_t\|^2$$

μ limite la dérive ; tolère un travail local variable (clients lents).
SCAFFOLD (Karimireddy et al., 2020) : variables de contrôle c (serveur) et c_k (client) corrigent le gradient local :

$$w \leftarrow w - \eta\left(\nabla F_k(w) - c_k + c\right)$$

Corrige la dérive, converge plus vite en non-IID, mais **double la communication** et exige des clients avec état.
FedNova : normalise les mises à jour par le nombre de pas locaux pour éviter un biais d'objectif.
Remarque sécurité : les variables de contrôle élargissent la surface d'attaque (un client malveillant peut les falsifier).

Personnalisation

- Un modèle unique est-il pertinent si les distributions diffèrent ? Alternative : **modèles personnalisés**.
- **Ajustement fin local** du modèle global (simple, souvent fort).
- **Couches partagées + têtes locales** (FedPer, LG-FedAvg).
- **Régularisation vers le global** (Ditto, pFedMe) : $\min F_k(v) + \frac{\lambda}{2}\|v - w\|^2$.
- **Clustering de clients** (IFCA) : un modèle par groupe de distributions similaires.
- **Méta-apprentissage** (Per-FedAvg).
- **Compromis** : personnalisation améliore la précision locale et peut aussi *atténuer* certaines attaques ou en *masquer* l'effet.

Participation partielle, compression, asynchronie, outils

Contraintes système

- **Coût de communication** : domine souvent le coût de calcul. Un modèle de $d = 10^7$ paramètres en float32 pèse 40 Mo par client et par tour (montée + descente).
- **Participation partielle** : seule une fraction C des clients participe à chaque tour ; échantillonnage biaisé par la disponibilité (appareil branché, wifi, nuit).
- **Retardataires** (*stragglers*) : délais, abandons ; stratégies de *timeout*, sur-échantillonnage, asynchronie.
- **Hétérogénéité matérielle** : calcul, mémoire, énergie variables.
- **Hétérogénéité logicielle** : versions, plate-formes.

Compression des mises à jour

Technique	Principe	Remarque
Quantification	Réduire la précision (8 bits, 1 bit, QSGD)	Biais maîtrisé
Sparsification top-k	N'envoyer que les k plus grandes coordonnées	Nécessite compensation d'erreur
Compensation d'erreur	Accumuler le résidu non envoyé	Améliore la convergence
Sketching	Projections aléatoires	Compatible avec agrégation linéaire
Distillation	Échanger des sorties plutôt que des poids	Réduit la taille ; autres fuites

Exemple : $d = 10^7$, top-1 % : 10^5 valeurs $\times$ (4 octets valeur + 4 octets indice) = 0,8 Mo au lieu de 40 Mo, soit un facteur 50.

FL asynchrone, hiérarchique et outils

- **Asynchrone** (FedAsync, FedBuff) : le serveur applique les mises à jour à l'arrivée ; pondération par l'obsolescence (*staleness*).
- **Hiérarchique** : agrégateurs intermédiaires (bords, régions) pour réduire la charge.
- **Cadres logiciels** : Flower, TensorFlow Federated, FedML, PySyft, FATE, OpenFL, NVIDIA FLARE.
- **Jeux de données de référence** : LEAF (FEMNIST, Shakespeare, Sent140), FedScale, partitions Dirichlet de CIFAR-10/100.
- **Bonne pratique de simulation** : fixer les graines, rapporter K, C, E, α, nombre de tours, et la variance sur plusieurs essais.

Fuites, confidentialité différentielle, agrégation sécurisée

Les mises à jour fuient de l'information

- **Reconstruction de données** : inversion de gradients (DLG, Geiping et al.) : retrouver entrées et étiquettes à partir d'un gradient.
- **Inférence d'appartenance** : un échantillon a-t-il servi à l'entraînement ? (Shokri et al.)
- **Inférence de propriétés** : détecter un attribut non lié à la tâche (ex. présence d'un type d'objet dans les données d'un client).
- **Extraction du modèle** et inversion de modèle sur le modèle final.
- **Plus c'est vulnérable** : petits lots, nombreuses époques locales, grands modèles qui mémorisent, serveur curieux voyant chaque mise à jour individuelle.

Exemple : fuite exacte pour un lot de taille 1

Couche dense $z = Wx + b$, perte ℓ, $\delta = \partial \ell / \partial z$. Alors

$$\frac{\partial \ell}{\partial W} = \delta\, x^\top, \qquad \frac{\partial \ell}{\partial b} = \delta$$

Pour une ligne i avec $\delta_i \neq 0$:

$$x = \frac{(\partial \ell / \partial W)_{i,\cdot}}{(\partial \ell / \partial b)_i}$$

- Le serveur lit le gradient (ou la mise à jour après un seul pas) et **retrouve l'entrée exactement**.
- Avec un lot de B échantillons, le gradient est une somme : le problème devient sous-déterminé mais reste résoluble par optimisation (DLG) pour de petits lots et petits réseaux.
- **Leçon** : transmettre des gradients revient à transmettre des informations sur les données.

Confidentialité différentielle (DP)

Mécanisme $\mathcal{M}$ est (ε, δ)-DP si pour tout couple de jeux voisins D, D' et tout ensemble O :
$\Pr[\mathcal{M}(D) \in O] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in O] + \delta$.
DP au niveau client (DP-FedAvg) :

1. Écrêtage de chaque mise à jour : $\Delta_k \leftarrow \Delta_k \cdot \min(1, C/\|\Delta_k\|)$.
2. Agrégation, puis ajout de bruit gaussien $\mathcal{N}(0, \sigma^2 C^2 I)$ à la somme.
3. Suivi du budget (ε, δ) par comptabilité (moments, Rényi) selon σ, taux d'échantillonnage q et nombre de tours T.

Exemple (mécanisme gaussien simple, sensibilité $\Delta = 1$, $\varepsilon = 0{,}5$, $\delta = 10^{-5}$) :
$\sigma \geq \Delta\sqrt{2\ln(1{,}25/\delta)}/\varepsilon \approx 4{,}85/0{,}5 \approx 9{,}7$.
Compromis : plus de bruit $\Rightarrow$ meilleure confidentialité, moins d'utilité ; le budget se consomme à chaque tour.

Agrégation sécurisée (Bonawitz et al., 2017)

Objectif : le serveur n'apprend que la **somme** $\sum_k x_k$, pas les x_k individuels.
Exemple à 3 clients : chaque paire (i, j) partage un secret s_{ij} (échange de clés) ; x_1, x_2, x_3 sont les mises à jour.

- Client 1 envoie $y_1 = x_1 + s_{12} + s_{13}$
- Client 2 envoie $y_2 = x_2 - s_{12} + s_{23}$
- Client 3 envoie $y_3 = x_3 - s_{13} - s_{23}$

$y_1 + y_2 + y_3 = x_1 + x_2 + x_3$: les masques s'annulent. Chaque y_i seul est uniformément aléatoire.
Défis : abandons de clients (partage de secrets de Shamir, seuil), coût $O(K^2)$ en cryptographie de paires, serveur malveillant (vérifiabilité), incompatibilité avec les défenses qui inspectent les mises à jour individuelles (module 7).
Autres outils : chiffrement homomorphe, calcul multipartite, TEE (SGX, SEV).

Combiner les protections et leurs limites

Mécanisme	**Protège contre**	**Ne protège pas**
FL seul	Transfert brut des données	Fuites via gradients
Agrégation sécurisée	Serveur curieux (voit la somme)	Inférence sur la somme, modèle final
DP client	Inférence d'appartenance, mémorisation	Utilité réduite ; pas d'intégrité
Chiffrement homomorphe	Serveur curieux	Coût ; pas d'intégrité
TEE	Serveur non fiable	Confiance dans le matériel, canaux auxiliaires

Principe : combiner agrégation sécurisée (confidentialité vis-à-vis du serveur) et DP (garantie sur la sortie) ; la confidentialité n'apporte *aucune* garantie d'intégrité.

Poisoning, backdoors, Sybil, adaptivité

Taxonomie des attaques par empoisonnement

Critère	**Variantes**	**Exemple**
Objectif	Non ciblé / ciblé / backdoor	Dégrader la précision / classer une classe / déclencher sur un motif
Vecteur	Poisoning de données / de modèle	Étiquettes altérées / mise à jour forgée
Connaissance	Boîte blanche / grise / noire	Connaît ou non l'agrégation, les défenses
Capacité	Fraction de clients compromis α, nombre de tours actifs	1 % des clients, quelques tours
Furtivité	Mises à jour proches de la norme honnête	Contourne l'écrêtage et la détection

Mesures : précision sur la tâche principale (MA) et taux de succès de l'attaque (ASR).

Backdoors en FL : principe

- **Objectif** : le modèle se comporte normalement sauf si l'entrée contient un **déclencheur** (*trigger*), auquel cas il prédit une classe cible choisie par l'attaquant.
- **Mise en oeuvre** : le client malveillant entraîne localement sur un mélange de données propres et de données déclenchées étiquetées avec la classe cible, puis envoie sa mise à jour.
- **Variantes de trigger** : patch fixe, trigger distribué entre attaquants (DBA), trigger optimisé (A3FL, adaptatif au modèle global), *edge-case* (Attack of the Tails).
- **Persistance** : le défi est de survivre à la dilution par les mises à jour honnêtes ; techniques : mise à l'échelle, projection dans une région peu utilisée, mises à jour dans des coordonnées rarement modifiées (Neurotoxin).

Remplacement de modèle : exemple chiffré

Le serveur applique $G_{t+1} = G_t + \dfrac{\eta}{m} \sum_{i=1}^{m} (L_i - G_t)$ avec m clients par tour.
L'attaquant veut imposer un modèle X (backdoorisé). Il envoie

$$\tilde{L} = G_t + \gamma\,(X - G_t), \qquad \gamma = \frac{m}{\eta}$$

Alors $G_{t+1} = G_t + \frac{\eta}{m}\Big[\gamma(X - G_t) + \sum_{\text{honnêtes}} (L_i - G_t)\Big] = X + \frac{\eta}{m} \sum_{\text{honnêtes}} (L_i - G_t)$.
Exemple : $m = 100$, $\eta = 1$, $\gamma = 100$; si le modèle global a presque convergé, les $L_i - G_t$ honnêtes sont petits : $G_{t+1} \approx X$.
Contre-mesure évidente : la norme $\|\tilde{L} - G_t\| = \gamma \|X - G_t\|$ est grande, d'où l'écrêtage. L'attaquant adaptatif choisit alors γ de sorte que la norme reste sous le seuil, au prix d'une persistance plus faible.

Sybil, collusion, free-riding et attaques adaptatives

- **Sybil** : multiplier les identités pour dépasser le seuil de tolérance de l'agrégateur ; plus facile en cross-device (comptes bon marché).
- **Collusion** : plusieurs attaquants coordonnent leurs mises à jour pour rester proches de la moyenne tout en poussant dans une direction (attaques de type *little is enough*).
- **Free-riding** : un client sans données envoie du bruit ou le modèle reçu pour bénéficier du modèle.
- **Attaques adaptatives** : l'attaquant connaît la défense et optimise contre elle ; l'évaluation d'une défense sans attaquant adaptatif est insuffisante.
- **Observation de recherche** : l'efficacité des attaques dépend fortement de α (non-IID), du taux de compromission et de la fréquence d'activité.

Agrégateurs robustes, détection, écrêtage, limites

Agrégateurs robustes : médiane et moyenne tronquée

Sur chaque coordonnée, on remplace la moyenne par une statistique robuste.
Exemple (5 clients, une coordonnée) : valeurs (1,0; 1,1; 0,9; 1,0; 50).

- Moyenne : $(1 + 1{,}1 + 0{,}9 + 1 + 50)/5 = 10{,}8$ (détruite par l'outlier).
- Médiane : 1,0.
- Moyenne tronquée ($\beta = 1$ de chaque côté) : trier $(0{,}9; 1{,}0; 1{,}0; 1{,}1; 50)$, retirer 0,9 et 50, moyenne de $(1{,}0; 1{,}0; 1{,}1) \approx 1{,}03$.

Garanties (Yin et al., 2018) : erreur d'ordre $O(\alpha/\sqrt{n}) + \ldots$ sous hypothèses de bornage et d'i.i.d. ; **se dégrade en non-IID** car des clients honnêtes mais atypiques sont écartés.
Coût : tri par coordonnée $O(md \log m)$.

Krum, Multi-Krum et géométrie des mises à jour

Krum (Blanchard et al., 2017) : pour m clients dont f byzantins, avec $m \geq 2f + 3$. Pour chaque mise à jour Δ_i, score

$$s(i) = \sum_{j \in \mathcal{N}_i} \|\Delta_i - \Delta_j\|^2, \qquad \mathcal{N}_i = \text{les } m - f - 2 \text{ plus proches voisins}$$

Sélectionner la mise à jour de score minimal (Multi-Krum : moyenne des q meilleures).

- **Hypothèse implicite** : les mises à jour honnêtes sont proches les unes des autres.
- **Faiblesses** : non-IID (honnêtes éloignés), attaques qui se placent près du centre, sélection d'un seul client (perte de données), coût $O(m^2 d)$.
- **Variantes** : Bulyan, RFA (médiane géométrique).

Écrêtage, détection et défenses avec données de référence

- **Écrêtage de norme** + éventuellement bruit (*weak DP*) : limite l'influence d'un client, atténue la mise à l'échelle.
- **FLTrust** : le serveur dispose d'un petit jeu propre ; pondère les mises à jour par la similarité cosinus avec son gradient de référence.
- **FLAME** : regroupement des mises à jour (cosinus), écrêtage adaptatif, bruit calibré pour neutraliser les backdoors.
- **Analyse de neurones / couches** : détecter des écarts sur les couches finales ou les activations.
- **Défenses post-hoc** : élagage, ajustement fin sur données propres, détection de triggers (Neural Cleanse adapté).
- **Défenses certifiées** : garanties de robustesse sous un nombre borné de clients malveillants (agrégation par vote, lissage).

Limites des défenses et tension avec la vie privée

Défaut	Cause	Conséquence
Défense inefficace en non-IID	Honnêtes dispersés	Faux positifs, perte de clients utiles
Attaque adaptative	Connaissance de la défense	Contournement
Conflit avec l'agrégation sécurisée	Inspection individuelle impossible	Choisir entre vie privée et inspection
Hypothèse de majorité honnête	Sybil	Rupture des garanties
Dépendance à un jeu de référence	Biais, disponibilité	Défense contournable ou biaisée

Pistes : agrégation sécurisée avec vérification de bornes (preuves ZK de norme), écrêtage vérifiable, défense en profondeur (identités, quotas, détection, robustesse), évaluation face à des attaquants adaptatifs.

Gossip, topologie, consensus, blockchain, menaces sans serveur

Pourquoi décentraliser ?

- Supprimer le **serveur central** : point de défaillance, goulot de communication, point de confiance.
- **DFL** : chaque client échange avec ses voisins dans un graphe de communication $G = (V, E)$ et met à jour son modèle local.
- **Cas d'usage** : réseaux de consortium sans acteur neutre, appareils en bordure, réseaux véhiculaires, contextes de souveraineté.
- **Coût** : convergence plus lente, hétérogénéité des modèles locaux, gouvernance et sécurité plus difficiles.

Gossip et moyennage de consensus

Modèles x_i^t ; matrice de mélange W symétrique, **doublement stochastique** (lignes et colonnes de somme 1) :

$$x^{t+1} = Wx^t \quad \text{(moyennage)}, \qquad \text{D-PSGD} : x_i^{t+1} = \sum_j W_{ij} x_j^t - \eta \nabla F_i(x_i^t)$$

Exemple : anneau de 4 noeuds, chaque noeud pondère 1/3 lui-même et 1/3 chacun de ses deux voisins. Valeurs initiales $(4, 0, 0, 0)$.

$$x^1 = \left(\tfrac{4}{3},\ \tfrac{4}{3},\ 0,\ \tfrac{4}{3}\right), \quad \text{tend vers la moyenne 1}$$

Vitesse : gouvernée par $\lambda_2(W)$, la deuxième plus grande valeur propre en module ; $|\lambda_2|$ proche de 1 $\Rightarrow$ convergence lente (graphes peu connectés).
Compromis topologie : anneau (peu de liens, lent), graphe complet (rapide, coûteux), graphes aléatoires/expanseurs (bon équilibre).

Menaces dans le FL décentralisé

- **Pas de vue globale** : chaque noeud ne voit que ses voisins ; la détection de comportement anormal est locale.
- **Voisins byzantins** : un noeud malveillant envoie des modèles différents à chacun de ses voisins (équivocation).
- **Propagation** : une backdoor injectée localement se diffuse via le gossip ; la topologie détermine la vitesse et la portée.
- **Position de l'attaquant** : un noeud central dans le graphe a un impact disproportionné.
- **Observabilité** : sans serveur, il est difficile de mesurer l'état global de l'attaque ; analyser les effets via des estimations locales.
- **Défenses** : agrégation robuste locale (médiane/tronquée entre voisins), pondération par la confiance ou la similarité, sélection de voisins, authentification des pairs.

Blockchain et FL

- **Rôles possibles** : registre d'audit des mises à jour (hachages), coordination décentralisée des tours, gestion des incitations, identité et admission des participants.
- **Ce qui reste hors chaîne** : modèles et mises à jour (taille), données.
- **Apports** : traçabilité, intégrité de l'historique, tolérance à la panne du coordinateur, règles transparentes.
- **Limites** : latence et coût, ne garantit pas la qualité des mises à jour, ne remplace ni l'agrégation robuste ni la vie privée, charge de consensus.
- **Réseaux de consortium** (ex. Besu en QBFT) : adaptés au cross-silo avec vérification des participants.
- Question de conception : la blockchain est-elle nécessaire ou un serveur audité suffit-il ?

Contribution, valorisation, RGPD, AI Act, désapprentissage

Équité et valeur des contributions

- **Équité de performance** : le modèle global peut être moins bon pour des clients minoritaires ; viser la **variance** de précision entre clients (q-FFL, AFL).
- **Valeur de contribution** : valeur de Shapley d'un client i :

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} \big(v(S \cup \{i\}) - v(S)\big)$$

  coûteuse ($2^{|N|}$) ; approximations par échantillonnage, gradients, influence.
- **Incitations** : paiements, accès au modèle, réputation, mécanismes de théorie des jeux ; risques : manipulation de la mesure de contribution, duplication d'identités.
- **Free-riding** : détecter la contribution nulle ou bruitée.

Cadre juridique et conformité

- **RGPD** : minimisation, finalité, base légale, rôles (responsable conjoint ou sous-traitant), droit à l'effacement, analyse d'impact.
- Les mises à jour de modèle peuvent constituer des **données personnelles** si une ré-identification est possible.
- **AI Act** (UE) : exigences de gestion des données, documentation, supervision humaine, robustesse et cybersécurité pour les systèmes à haut risque.
- **Secteur de la santé** : cadres nationaux, accords inter-établissements, espaces de données de santé.
- **Bonne pratique** : DPIA, contrats de gouvernance de la fédération, journalisation, évaluation de la vie privée avec attaques réelles.
- **Désapprentissage fédéré** : retirer l'influence d'un client (droit à l'effacement) sans tout ré-entraîner ; difficile et aux garanties limitées.

Évaluer correctement un système FL

Dimension	**Métriques**	**Précautions**
Utilité	Précision globale, par client, variance	Jeu de test représentatif, pas seulement agrégé
Communication	Octets par tour, tours jusqu'à cible	Compter montée et descente
Vie privée	(ε, δ), succès d'attaques d'inférence	Attaquants réalistes, pas seulement bornes théoriques
Robustesse	MA, ASR selon α et fraction compromise	Attaques adaptatives ; plusieurs hétérogénéités
Coûts	Calcul, énergie, temps mur	Matériel de référence
Reproductibilité	Graines, hyperparamètres, partitions	Publier le code et les partitions

Imagerie médicale inter-hôpitaux et conception sécurisée

Cas : détection de tumeurs sur imagerie inter-hôpitaux

- **Contexte** : 8 hôpitaux, scanners hétérogènes, populations différentes (non-IID de caractéristiques et d'étiquettes).
- **Architecture** : cross-silo, FedAvg avec FedProx pour la dérive ; personnalisation par ajustement fin local.
- **Vie privée** : agrégation sécurisée + DP au niveau client ; accès au modèle restreint ; audit d'inférence d'appartenance avant publication.
- **Intégrité** : identification forte des clients (PKI), écrêtage de norme, agrégation robuste, jeu de validation indépendant, détection de dérive d'ASR sur triggers connus.
- **Gouvernance** : contrat de fédération, journal d'audit (éventuellement ancré sur un registre de consortium), responsabilités RGPD.
- **Question** : que se passe-t-il si un hôpital est compromis ou malveillant ?

Scénarios d'attaque et mesures

Attaque	Effet	Mesure
Hôpital malveillant (backdoor)	Fausse classification sur motif	Écrêtage, agrégation robuste, audit de triggers
Serveur curieux	Reconstruction d'images	Agrégation sécurisée, DP, lots plus grands
Collusion de clients	Inférence sur un autre hôpital	Seuil d'agrégation, DP
Inférence d'appartenance sur le modèle	Fuite de la présence d'un patient	DP, régularisation, limitation d'accès
Clé compromise	Usurpation d'un hôpital	Rotation, révocation, authentification mutuelle
Déni de service	Tours bloqués	Timeouts, quorum minimal, redondance

Arbitrages de conception

Axe	Option A	Option B
Topologie	Serveur central (simple, contrôle)	DFL (pas de point central, plus de risques locaux)
Vie privée	Agrégation sécurisée (pas de DP)	DP (garantie formelle, perte d'utilité)
Robustesse	Agrégateur robuste (inspection)	Agrégation sécurisée (opacité)
Hétérogénéité	Modèle global (simple)	Personnalisation (meilleur local)
Communication	Compression forte (coût réduit)	Modèle complet (précision)
Confiance	Serveur audité	Registre de consortium

Aucune option dominante : justifier chaque choix par le modèle de menaces, la réglementation et les ressources.

Exercices (1/2) : analyse

1. Quatre clients de tailles $(50, 150, 300, 500)$ envoient les scalaires $(2,\ 4,\ 1,\ 3)$. Calculer l'agrégat FedAvg pondéré et l'effet si le client de taille 50 déclare 5000.
2. Montrer qu'avec $E = 1$, un seul pas de SGD par client et des lots complets, FedAvg équivaut à un pas de gradient centralisé.
3. Reproduire l'exemple d'agrégation sécurisée à 3 clients avec $x = (5, 7, 9)$ et $s_{12} = 3$, $s_{13} = 8$, $s_{23} = 2$; vérifier la somme et expliquer pourquoi y_1 seul ne révèle rien.
4. Un attaquant veut imposer X avec $m = 50$, $\eta = 1$. Calculer γ et la norme de sa mise à jour si $\|X - G_t\| = 0{,}4$; que se passe-t-il avec un écrêtage à $C = 2$?
5. Comparer médiane, moyenne tronquée et Krum face à 20 % d'attaquants colludant dans un contexte non-IID ($\alpha = 0{,}1$).

Exercices (2/2) : conception d'un système FL sécurisé

Énoncé. Un consortium de cinq banques veut entraîner un modèle de détection de fraude sans partager de transactions.

Livrables :

1. Choix de la taxonomie (cross-silo, horizontal/vertical) et justification.
2. Algorithme d'optimisation et gestion du non-IID.
3. Modèle de menaces (serveur, banques, externes) avec au moins six menaces.
4. Protections de vie privée (agrégation sécurisée, DP) et budget.
5. Défenses d'intégrité et compatibilité avec les protections de vie privée.
6. Option centralisée ou décentralisée (DFL, blockchain) ; justification.
7. Plan d'évaluation : métriques, attaquants adaptatifs, reproductibilité.
8. Conformité (RGPD, secret bancaire) et gouvernance.

Questions de discussion

- Le FL protège-t-il réellement la vie privée sans DP ni agrégation sécurisée ?
- Peut-on obtenir à la fois robustesse aux backdoors et agrégation sécurisée ?
- La personnalisation rend-elle les attaques moins efficaces, ou plus difficiles à détecter ?
- Un attaquant adaptatif peut-il toujours contourner une défense connue ? Quelles garanties certifiées sont réalistes ?
- Dans quels contextes la décentralisation (DFL) apporte-t-elle un vrai bénéfice de sécurité ?

Synthèse

- Le **FL** déplace le calcul vers les données ; FedAvg est la base, la **dérive en non-IID** son principal défi.
- Les **mises à jour fuient** des données : DP et agrégation sécurisée sont complémentaires.
- Les **backdoors** exploitent la participation ouverte ; la mise à l'échelle et les triggers adaptatifs rendent la défense difficile.
- Les **défenses robustes** souffrent du non-IID, des attaques adaptatives et de leur conflit avec l'agrégation sécurisée.
- Le **FL décentralisé** supprime le serveur mais déplace la confiance vers la topologie et les voisins.
- L'évaluation doit combiner utilité, communication, vie privée et robustesse, face à des attaquants réalistes.

Références

- McMahan B. et al., *Communication-Efficient Learning of Deep Networks from Decentralized Data*, AISTATS 2017.
- Kairouz P. et al., *Advances and Open Problems in Federated Learning*, 2021.
- Li T. et al., *Federated Optimization in Heterogeneous Networks* (FedProx), MLSys 2020.
- Karimireddy S. et al., *SCAFFOLD*, ICML 2020.
- Bonawitz K. et al., *Practical Secure Aggregation for Privacy-Preserving ML*, CCS 2017.
- Zhu L., Liu Z., Han S., *Deep Leakage from Gradients*, NeurIPS 2019 ; Shokri R. et al., *Membership Inference Attacks*, S&P 2017.
- Abadi M. et al., *Deep Learning with Differential Privacy*, CCS 2016.
- Bagdasaryan E. et al., *How To Backdoor Federated Learning*, AISTATS 2020 ; Xie C. et al., *DBA*, ICLR 2020 ; Wang H. et al., *Attack of the Tails*, NeurIPS 2020.
- Blanchard P. et al., *Krum*, NeurIPS 2017 ; Yin D. et al., *Byzantine-Robust Distributed Learning*, ICML 2018.
- Cao X. et al., *FLTrust*, NDSS 2021 ; Nguyen T. et al., *FLAME*, USENIX Security 2022.
- Lian X. et al., *Can Decentralized Algorithms Outperform Centralized Algorithms ?*, NeurIPS 2017.
- Règlement RGPD ; règlement européen sur l'IA (AI Act).